

APB REAL-TIME CLOCK CORE

Synthesizable Calendar, Alarm, and Interrupt IP

PURPOSE

This document defines the externally visible behavior, functional architecture, programming model, clock and reset requirements, and verification intent of the APB Real-Time Clock Core. It is the integration baseline for RTL, verification, synthesis, and SoC teams.

DOCUMENT TYPE

Hardware Architecture Specification (HAS)

DOCUMENT ID

HAS-APB-RTC-001

REVISION

1.0

STATUS

Engineering release

DATE

18 July 2026

CLASSIFICATION

Internal technical documentation



IP FAMILY
Timing



INTERFACE
APB3



LANGUAGE
SystemVerilog



VERIFICATION
UVM Ready



LICENSE
MIT

OCX

OPENCORELABSX



TABLE OF CONTENTS

TABLE OF CONTENTS	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
REVISION HISTORY	vii
1 Product Overview	9
1.1 Scope	9
1.2 Design Objectives	9
1.3 Target Applications	9
1.4 Out of Scope	9
2 General Description	9
2.1 Features	9
2.2 Standards Compliance	10
2.3 Functional Description	10
2.4 Architecture	11
2.4.1 Controller State Flow	12
2.4.2 Tick, Alarm, and Interrupt Timing	13
2.5 Block Description	13
2.6 Integration Constraints	14
3 APB Interface Background	14
3.1 Terminology	15
3.2 Bus Transaction	15
3.3 Bus Protocol	15
4 Configuration Guide	15
4.1 Reset and Initialization	15
4.2 Clock Configuration	15



TABLE OF CONTENTS

Detailed Design / Architecture / TABLE OF CONTENTS

4.3 Read and Write Registers 16

4.4 Interrupts and Error Handling 16

5 Parameters 16

6 Signals Description 16

6.1 APB Slave Interface Signals 16

6.2 Interrupt Signals 17

7 Registers 17

7.1 Register Memory Map 17

7.2 Registers and Field Descriptions 18

7.2.1 CTRL 18

7.2.2 STATUS 18

7.2.3 PRESCALER 19

7.2.4 TIME 19

7.2.5 DATE 20

7.2.6 ALARM_TIME 20

7.2.7 ALARM_DATE 21

7.2.8 IRQ_EN 22

7.2.9 IRQ_STAT 22

7.2.10 VERSION 23

8 Software Flow 23

8.1 Initialization Sequence 23

8.2 Normal Operation 23

8.3 Error Recovery 24

9 Verification 25

9.1 Verification Environment 25

9.1.1 Top Testbench 25

9.1.2 APB Agent 25

9.1.3 Monitor 25

9.1.4 Scoreboard 25



LIST OF FIGURES

1	APB RTC functional architecture	11
2	RTC controller state flow	12
3	Illustrative RTC tick, calendar, alarm, and interrupt timing	13
4	Recommended APB RTC initialization and event-service flow	24



LIST OF TABLES

1	Document revision history	vii
2	Functional block responsibilities	14
3	Configurable design parameters	16
4	APB slave interface signals	16
5	Interrupt interface signals	17
6	APB register map	17
7	<i>RTC Control Register Field Description</i>	18
8	<i>RTC Status Register Field Description</i>	18
9	<i>RTC Prescaler Register Field Description</i>	19
10	<i>RTC Time Register Field Description</i>	19
11	<i>RTC Date Register Field Description</i>	20
12	<i>RTC Alarm Time Register Field Description</i>	21
13	<i>RTC Alarm Date Register Field Description</i>	21
14	<i>RTC Interrupt Enable Register Field Description</i>	22
15	<i>RTC Interrupt Status Register Field Description</i>	22
16	<i>RTC Version Register Field Description</i>	23



REVISION HISTORY

Table 1: Document revision history

Revision	Date	Author	Description
A	18 July 2026	Brian Bui	Initial reusable HAS release



Solution Brief

Description

The APB RTC is a compact real-time clock for FPGA and ASIC integration. It maintains binary time and Gregorian calendar fields, derives a one-second event from the APB clock, supports software load and readback, compares an optional date-qualified alarm, and reports sticky interrupt status.

Architecture

A single APB clock domain contains the register interface, programmable prescaler, calendar counters, alarm comparator, and interrupt state. No functional clock-domain crossing exists inside the IP.

Key Features

- Binary second, minute, hour, day, month, and 12-bit year counters
- Gregorian leap-year and month-length handling
- Programmable one-second prescaler
- Time/date load and readback
- Optional date-qualified alarm
- Sticky, write-one-to-clear interrupt status

Target Applications

- SoC timekeeping
- Low-complexity event scheduling
- FPGA control systems
- Always-on peripheral subsystems

Integration note

Technical information is subject to implementation review. System-level safety, security, entropy, retention, and cryptographic compliance remain the integrator's responsibility unless explicitly stated.



1 Product Overview

The APB RTC is a compact real-time clock for FPGA and ASIC integration. It maintains binary time and Gregorian calendar fields, derives a one-second event from the APB clock, supports software load and readback, compares an optional date-qualified alarm, and reports sticky interrupt status.

1.1 Scope

The specification covers `apb_rtc_wrapper`, its APB programming model, internal calendar behavior, alarm generation, and interrupt signaling.

1.2 Design Objectives

- Provide deterministic synthesizable timekeeping from PCLK.
- Expose a simple 32-bit APB software interface.
- Keep all software-visible status sticky until explicitly cleared.

1.3 Target Applications

- SoC timekeeping
- Low-complexity event scheduling
- FPGA control systems
- Always-on peripheral subsystems

1.4 Out of Scope

Battery-domain retention, oscillator accuracy, clock calibration, tamper resistance, and safety certification are system-level concerns.

2 General Description

2.1 Features

- Binary second, minute, hour, day, month, and 12-bit year counters
- Gregorian leap-year and month-length handling
- Programmable one-second prescaler
- Time/date load and readback
- Optional date-qualified alarm
- Sticky, write-one-to-clear interrupt status



2.2 Standards Compliance

The programming interface follows the APB setup/access transfer model and exposes 32-bit data, byte strobes, PREADY, and PSLVERR signals. Integration-level protocol compliance shall be confirmed against the AMBA APB revision selected by the SoC.

2.3 Functional Description

The RTL hierarchy is rooted at `apb_rtc_wrapper`, which instantiates one `apb_rtc_apb_interface`. The interface decodes the ten 32-bit registers, owns the programmable and sticky software-visible state, and instantiates `apb_rtc_core`. All logic executes in the `i_apb_rtc_pclk` domain; the design contains no internal clock-domain crossing.

The time base is a 32-bit up-counter. While `CTRL.ENABLE` is set and neither `TIME` nor `DATE` is being loaded, the combinational terminal condition is

$$w_second_tick = (prescale_cnt \geq PRESCALER). \quad (1)$$

The terminal edge clears the prescale counter and advances the calendar. Consequently, a stable value P in `PRESCALER` produces one RTC update every $P + 1$ PCLK cycles. A value of zero advances the RTC on every enabled PCLK edge. Disabling the RTC or writing `TIME` or `DATE` suppresses the terminal condition and clears the prescale counter.

Time and date are stored as unsigned binary fields, not BCD. Seconds and minutes cascade at 59, hours cascade at 23, and midnight updates the date. Month length is 31 days for January, March, May, July, August, October, and December; 30 days for April, June, September, and November; and 28 or 29 days for February. The leap-year predicate implements the Gregorian rule: divisible by four, except century years not divisible by 400.

The alarm comparator always compares hour, minute, and second. When `CTRL.ALARM_DATE_ENABLE` is set, year, month, and day must also match. The internal alarm event is a rising-edge detector on the complete enabled comparison, so a continuously equal calendar value generates one event rather than one event per PCLK cycle. `SECOND` and `ALARM` events set independent sticky `IRQ_STAT` bits. The interrupt output is the reduction-OR of `IRQ_STAT` masked by `IRQ_EN`.



2.4 Architecture

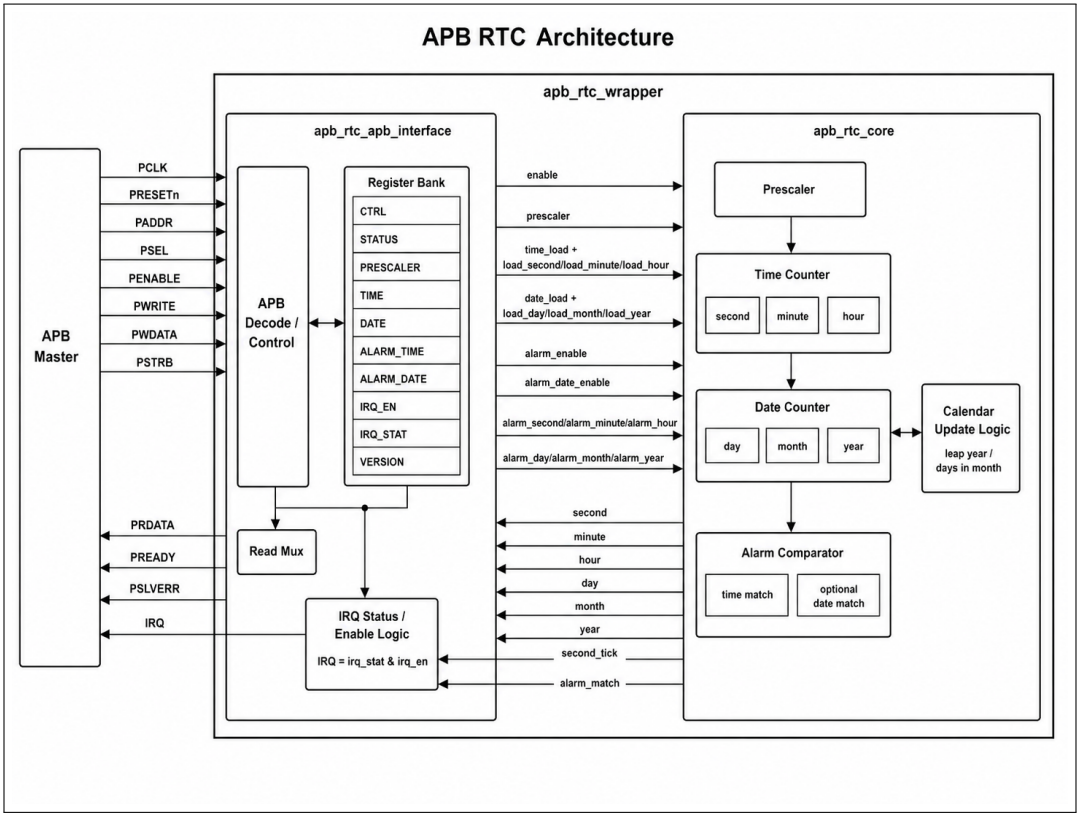


Figure 1: APB RTC functional architecture



2.4.1 Controller State Flow

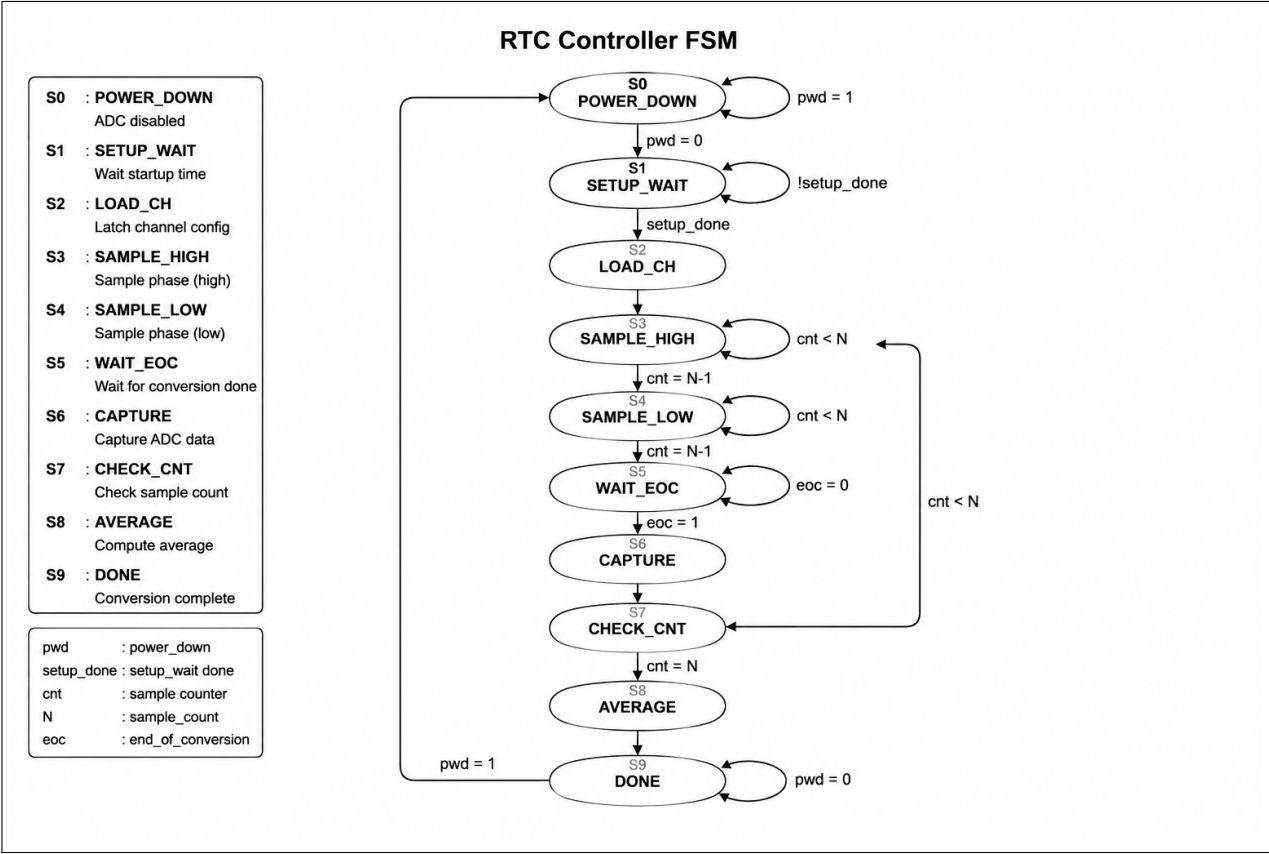


Figure 2: RTC controller state flow

Figure 2 provides the controller-state visualization supplied with the RTC design collateral. The normative timekeeping behavior remains defined by the synchronous counter and rollover logic described above.



2.4.2 Tick, Alarm, and Interrupt Timing

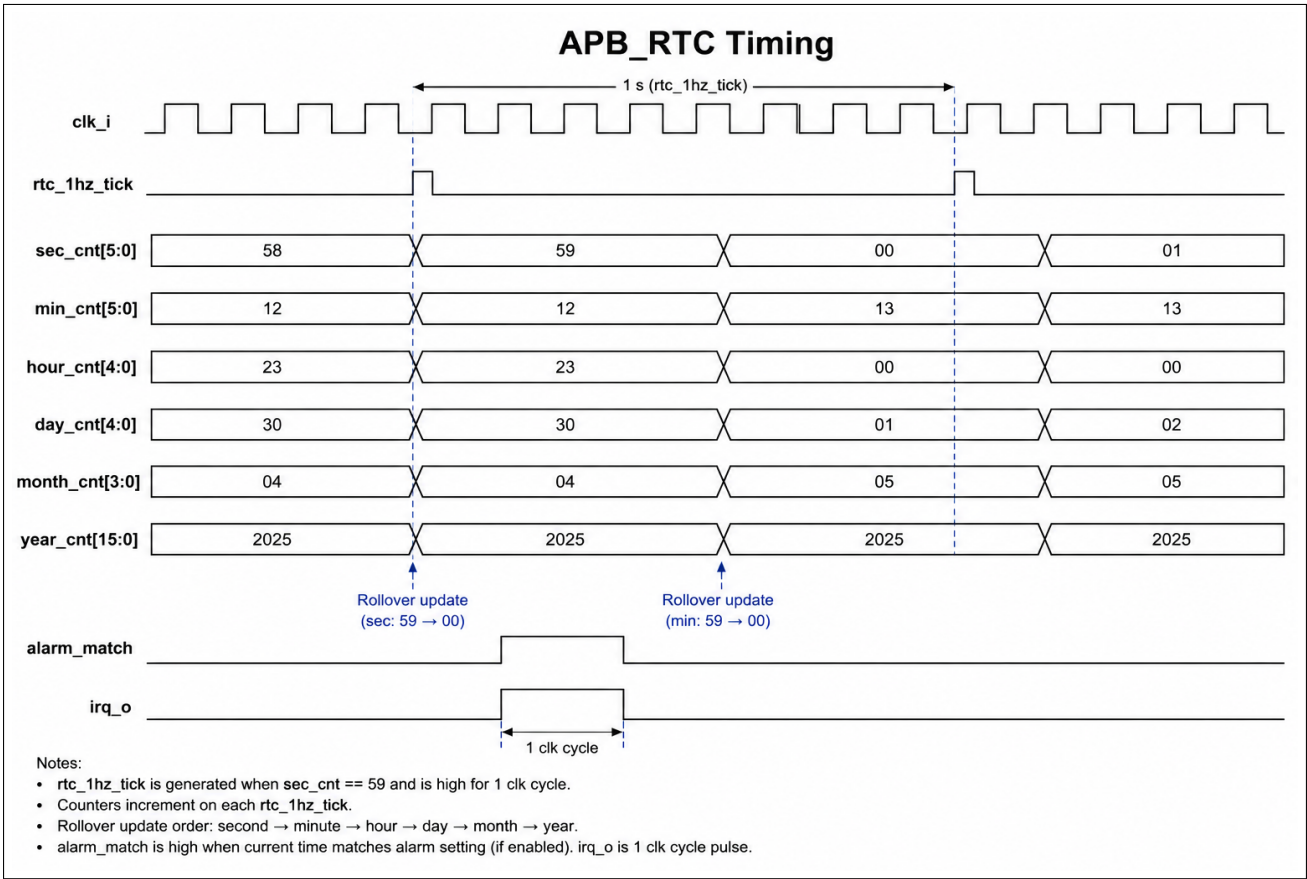


Figure 3: Illustrative RTC tick, calendar, alarm, and interrupt timing

Figure 3 uses `PRESCALER=2` for readability. The terminal condition is sampled at a `PCLK` edge to update the calendar and set the `SECOND` status. When that update first makes the alarm comparison true, the alarm-match pulse is visible during the following cycle and the sticky `ALARM` status is set on the next `PCLK` edge.

Event-set priority

`IRQ_STAT` is write-one-to-clear. If software clears a bit in the same cycle that the corresponding hardware event is asserted, the event assignment sets the bit and the event is retained.

2.5 Block Description



Table 2: Functional block responsibilities

Block	Responsibility	Implementation notes
apb_rtc_wrapper	Integration top	Exposes the APB and IRQ ports and instantiates the register interface.
apb_rtc_apb_interface	Programming model	Decodes writes, multiplexes reads, owns control, alarm, and IRQ registers, and instantiates the RTC core.
Prescaler	Time base	Counts PCLK edges while enabled and produces a terminal condition when the count is greater than or equal to PRESCALER.
Time counters	Binary timekeeping	Maintains second, minute, and hour and applies 59/59/23 rollover.
Date counters	Gregorian calendar	Maintains day, month, and 12-bit year and applies month-length and leap-year rollover at midnight.
Alarm comparator	Edge-detected event	Compares time and optional date fields and pulses only on a false-to-true match transition.
IRQ logic	Event reporting	Latches SECOND and ALARM events, implements write-one-to-clear behavior, and masks status with IRQ_EN.

2.6 Integration Constraints

Connect all APB signals to one PCLK domain, set PCLK_FREQ_HZ to the implemented clock rate, and route IRQ to the interrupt controller. The APB interconnect shall perform base-address decode because the IP consumes only its local address. Software must use full-word writes with the current RTL. Any requirement to retain time across main-power loss requires an always-on supply, retention strategy, or software restoration source outside this IP.

Integration constraint

The RTC derives time from PCLK and does not guarantee absolute clock accuracy. Main-power loss destroys time unless external retention is provided.

3 APB Interface Background



3.1 Terminology

Setup phase	The first cycle of an APB transfer, asserted with PSEL and PENABLE low.
Access phase	The following cycle, asserted with PSEL and PENABLE high until PREADY completes the transfer.
Subordinate	The IP receiving APB requests and returning PRDATA, PREADY, and PSLVERR.
Register offset	Byte address relative to the IP APB base address.

3.2 Bus Transaction

An APB transaction starts with a setup phase and advances to an access phase on the next PCLK edge. Address, direction, and write data remain stable through completion. Read data and error status are sampled when PSEL, PENABLE, and PREADY are asserted together.

3.3 Bus Protocol

The slave follows a single-clock APB access model. PREADY is continuously asserted, so an access phase completes without inserted wait states. Register writes occur only when PSEL, PENABLE, and PWRITE are asserted. PRDATA is combinational selected whenever PSEL is asserted for a read and is sampled during the completing access phase. The current RTL returns zero for an unmapped read, ignores an unmapped write, and permanently drives PSLVERR low. PSTRB is exposed at the wrapper boundary but is not consumed by the write logic; every accepted write supplies a complete 32-bit word.

4 Configuration Guide

4.1 Reset and Initialization

All functional state uses `i_apb_rtc_pclk`. The active-low asynchronous reset initializes the calendar to 2000-01-01 00:00:00, sets PRESCALER to `PCLK_FREQ_HZ-1`, initializes ALARM_DATE to 2000-01-01 and ALARM_TIME to 00:00:00, and clears CTRL, IRQ_EN, IRQ_STAT, the prescale counter, and alarm-edge history. Reset deassertion must meet the SoC reset strategy.

4.2 Clock Configuration

For an integer PCLK frequency f_{PCLK} and a desired RTC tick frequency f_{RTC} , program $PRESCALER = (f_{PCLK}/f_{RTC}) - 1$. The default parameter assumes a 1 Hz calendar tick. Fractional division, drift compensation, oscillator calibration, and dynamic clock-frequency detection are not implemented. Disable the RTC before changing clock or prescaler configuration to obtain a deterministic restart phase.



4.3 Read and Write Registers

Time, date, and alarm values use unsigned binary encoding, not BCD. All accepted writes are full-word operations because the current RTL does not apply PSTRB. TIME and DATE are live calendar load ports rather than ordinary storage registers: their write data is validated by the core, while reads return the current calendar state. Writing either register suppresses a tick in that cycle and clears the prescale counter. IRQ_STAT implements write-one-to-clear semantics with hardware-set priority.

4.4 Interrupts and Error Handling

The core exposes no APB error response: PSLVERR is tied low, invalid addresses read as zero, and invalid writes are ignored. Software shall validate the address map and account for full-word writes. TIME/DATE loads correct out-of-range fields as documented rather than reporting an error; alarm fields are not corrected. The IRQ output remains asserted while any enabled sticky status bit is set.

5 Parameters

Table 3: Configurable design parameters

Parameter	Default	Description
C_APB_DATA_WIDTH	32	APB data width.
C_APB_ADDR_WIDTH	8	APB address width.
PCLK_FREQ_HZ	50,000,000	Default APB frequency used for prescaler reset.

6 Signals Description

6.1 APB Slave Interface Signals

Table 4: APB slave interface signals

Signal	Direction	Width	Description
i_apb_rtc_pclk	Input	1	APB and RTC clock.
i_apb_rtc_presetn	Input	1	Active-low asynchronous reset.
i_apb_rtc_paddr	Input	8	APB byte address.
i_apb_rtc_psel	Input	1	Peripheral select.



Signal	Direction	Width	Description
i_apb_rtc_penable	Input	1	APB access phase.
i_apb_rtc_pwrite	Input	1	Write control.
i_apb_rtc_pwdata	Input	32	Write data.
i_apb_rtc_pstrb	Input	4	Byte strobes; exposed but not applied by the current register-write logic.
o_apb_rtc_prdata	Output	32	Read data.
o_apb_rtc_pready	Output	1	Transfer ready.
o_apb_rtc_pslverr	Output	1	Slave error, tied low.

6.2 Interrupt Signals

Table 5: Interrupt interface signals

Signal	Direction	Width	Description
o_apb_rtc_irq	Output	1	Combined enabled interrupt request.

7 Registers

7.1 Register Memory Map

Table 6: APB register map

Offset	Register	Access	Description
0x00	CTRL	R/W	Enable, alarm enable, and alarm-date enable.
0x04	STATUS	R	Running and pending-event status.
0x08	PRESCALER	R/W	APB clock cycles minus one per RTC second.
0x0C	TIME	R/W	Hour [20:16], minute [13:8], second [5:0].
0x10	DATE	R/W	Year [27:16], month [15:12], day [8:4].
0x14	ALARM_TIME	R/W	Alarm time using the TIME encoding.
0x18	ALARM_DATE	R/W	Alarm date using the DATE encoding.
0x1C	IRQ_EN	R/W	Second-tick and alarm enable bits.
0x20	IRQ_STAT	R/WIC	Sticky second-tick and alarm status.



Offset	Register	Access	Description
0x24	VERSION	R	IP version constant.

7.2 Registers and Field Descriptions

7.2.1 CTRL

Name: RTC Control Register

Size: 32 bits

Address offset: 0x00

Software access: Read/Write

The reset value is 0x00000000.

Table 7: *RTC Control Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:3	R/W	Reset 0. Stored and read back; no functional effect.
ALARM_DATE_ENABLE	2	R/W	Reset 0. Requires year, month, and day equality in addition to the alarm-time match.
ALARM_ENABLE	1	R/W	Reset 0. Enables the alarm comparator and edge detector.
ENABLE	0	R/W	Reset 0. Enables the prescaler and calendar counters.

7.2.2 STATUS

Name: RTC Status Register

Size: 32 bits

Address offset: 0x04

Software access: Read-only

The reset value is 0x00000000. Reading STATUS does not clear pending events.

Table 8: *RTC Status Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:3	R	Reset 0. Read zero.



Register Field	Bits	R/W	Description
ALARM	2	R	Reset 0. Mirrors sticky IRQ_STAT.ALARM.
SECOND	1	R	Reset 0. Mirrors sticky IRQ_STAT.SECOND.
RUNNING	0	R	Reset 0. Live reflection of CTRL.ENABLE.

7.2.3 PRESCALER

Name: RTC Prescaler Register

Size: 32 bits

Address offset: 0x08

Software access: Read/Write

The reset value is PCLK_FREQ_HZ-1.

Table 9: *RTC Prescaler Register Field Description*

Register Field	Bits	R/W	Description
DIVIDER	31:0	R/W	Selects one RTC update every DIVIDER+1 PCLK cycles. A write does not clear the active prescale counter.

7.2.4 TIME

Name: RTC Time Register

Size: 32 bits

Address offset: 0x0C

Software access: Read/Write load/readback

The reset value is 0x00000000. A write loads all time fields and restarts the prescaler phase.

Table 10: *RTC Time Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:21	R	Read zero; write bits ignored.
HOUR	20:16	R/W	Unsigned binary hour, legal range 0–23. An invalid load becomes 0.
Reserved	15:14	R	Read zero; write bits ignored.



Register Field	Bits	R/W	Description
MINUTE	13:8	R/W	Unsigned binary minute, legal range 0–59. An invalid load becomes 0.
Reserved	7:6	R	Read zero; write bits ignored.
SECOND	5:0	R/W	Unsigned binary second, legal range 0–59. An invalid load becomes 0.

7.2.5 DATE

Name: RTC Date Register

Size: 32 bits

Address offset: 0x10

Software access: Read/Write load/readback

The reset value encodes 2000-01-01. A write loads all date fields and restarts the prescaler phase.

Table 11: *RTC Date Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:28	R	Read zero; write bits ignored.
YEAR	27:16	R/W	Unsigned 12-bit year. The complete field is loaded without range correction.
MONTH	15:12	R/W	Unsigned month, legal range 1–12. An invalid load becomes 1.
Reserved	11:9	R	Read zero; write bits ignored.
DAY	8:4	R/W	Legal range is 1 through the Gregorian length of the supplied month/year; invalid input becomes 1.
Reserved	3:0	R	Read zero; write bits ignored.

7.2.6 ALARM_TIME

Name: RTC Alarm Time Register

Size: 32 bits

Address offset: 0x14

Software access: Read/Write

The reset value is 0x00000000. Alarm fields are stored without range correction.



Table 12: *RTC Alarm Time Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:21	R/W	Stored and read back; not used by the comparator.
HOUR	20:16	R/W	Alarm hour compared against the current HOUR field.
Reserved	15:14	R/W	Stored and read back; not used by the comparator.
MINUTE	13:8	R/W	Alarm minute compared against the current MINUTE field.
Reserved	7:6	R/W	Stored and read back; not used by the comparator.
SECOND	5:0	R/W	Alarm second compared against the current SECOND field.

7.2.7 ALARM_DATE

Name: RTC Alarm Date Register

Size: 32 bits

Address offset: 0x18

Software access: Read/Write

The reset value encodes 2000-01-01. Alarm fields are stored without range correction.

Table 13: *RTC Alarm Date Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:28	R/W	Stored and read back; not used by the comparator.
YEAR	27:16	R/W	Alarm year used when CTRL.ALARM_DATE_ENABLE is set.
MONTH	15:12	R/W	Alarm month used when CTRL.ALARM_DATE_ENABLE is set.
Reserved	11:9	R/W	Stored and read back; not used by the comparator.
DAY	8:4	R/W	Alarm day used when CTRL.ALARM_DATE_ENABLE is set.



Register Field	Bits	R/W	Description
Reserved	3:0	R/W	Stored and read back; not used by the comparator.

7.2.8 IRQ_EN

Name: RTC Interrupt Enable Register

Size: 32 bits

Address offset: 0x1C

Software access: Read/Write

The reset value is 0x00000000. Masking a source does not prevent its sticky status from being set.

Table 14: *RTC Interrupt Enable Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:2	R/W	Stored and read back; does not affect IRQ generation.
ALARM_ENABLE	1	R/W	Enables IRQ assertion when IRQ_STAT.ALARM is set.
SECOND_ENABLE	0	R/W	Enables IRQ assertion when IRQ_STAT.SECOND is set.

7.2.9 IRQ_STAT

Name: RTC Interrupt Status Register

Size: 32 bits

Address offset: 0x20

Software access: Read/Write-one-to-clear

The reset value is 0x00000000. A concurrent hardware event has priority over software clear.

Table 15: *RTC Interrupt Status Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:2	R/WIC	Read zero. Writing one has no observable effect.
ALARM	1	R/WIC	Sticky alarm event. Write one to clear; a concurrent alarm event sets it again.



Register Field	Bits	R/W	Description
SECOND	0	R/WIC	Sticky registered second-tick event. Write one to clear; a concurrent tick sets it again.

7.2.10 VERSION

Name:	RTC Version Register
Size:	32 bits
Address offset:	0x24
Software access:	Read-only

Table 16: *RTC Version Register Field Description*

Register Field	Bits	R/W	Description
VERSION	31:0	R	Constant 0x00010000, corresponding to RTL version 1.0.

8 Software Flow

8.1 Initialization Sequence

1. Hold PRESETn low while PCLK is stable, then release reset according to the SoC reset policy.
2. Confirm VERSION=0x00010000 and CTRL.ENABLE=0.
3. Program PRESCALER for the actual PCLK frequency and desired calendar tick.
4. Write a valid DATE word followed by a valid TIME word. Each load resets the prescaler, so the final TIME write defines the start of the first full interval.
5. If required, program ALARM_DATE and ALARM_TIME, then select time-only or date-qualified comparison in CTRL.
6. Clear stale SECOND and ALARM status by writing 0x00000003 to IRQ_STAT, then program IRQ_EN.
7. Set CTRL.ENABLE while preserving the selected alarm control bits.

8.2 Normal Operation

1. Poll STATUS or wait for o_apb_rtc_irq. RUNNING only indicates CTRL.ENABLE; it is not an event indication.
2. Read IRQ_STAT to distinguish SECOND from ALARM. Both bits may be set simultaneously and shall be serviced independently.



3. Read TIME and DATE when a calendar snapshot is required. Because the two registers are read in separate APB transfers, software should re-read TIME if rollover consistency across midnight is critical.
4. Service the pending source, then write one to each handled IRQ_STAT bit. Leave unrelated bits zero in the WIC value.
5. Continue with CTRL.ENABLE asserted. Reprogramming TIME or DATE restarts the prescaler interval and may change the alarm comparison immediately.

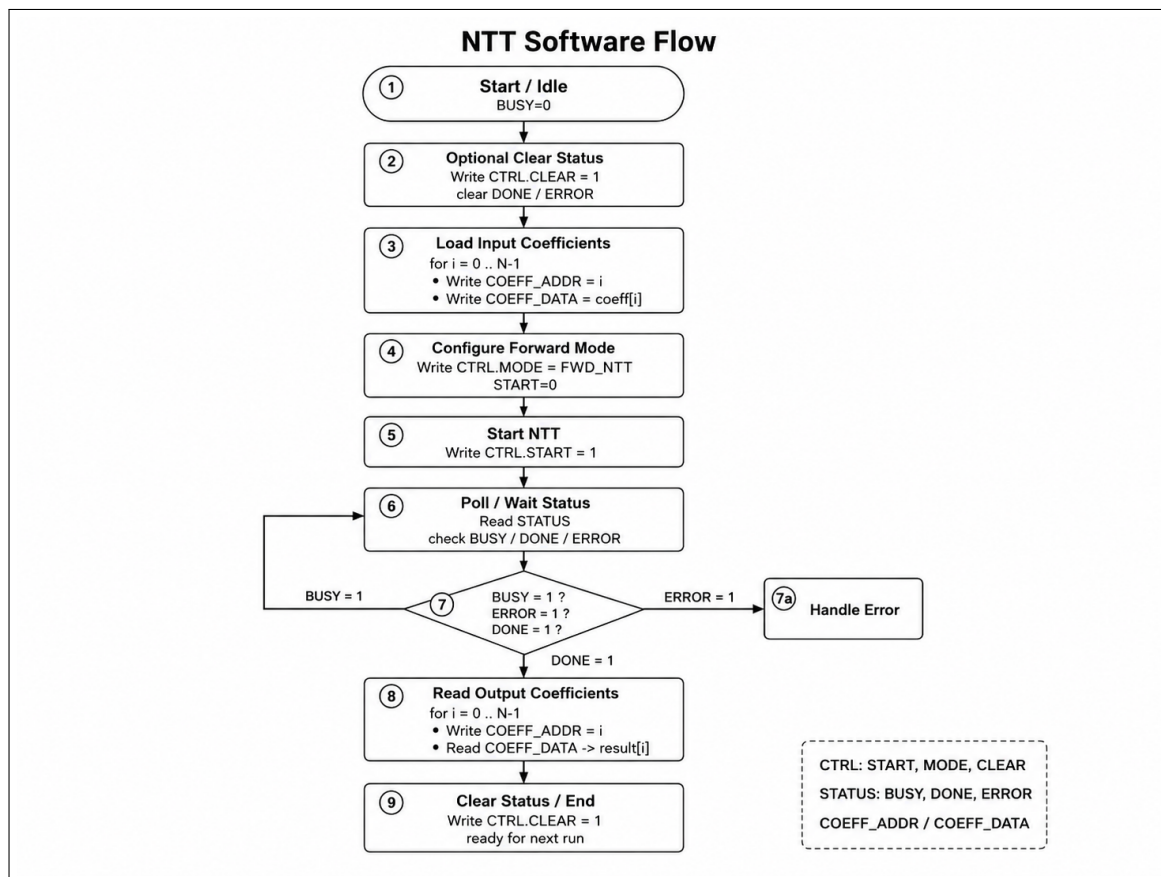


Figure 4: Recommended APB RTC initialization and event-service flow

Figure 4 summarizes the programming order and interrupt-service loop. Alarm registers should be initialized before ALARM_ENABLE is asserted to prevent an unintended false-to-true comparison transition.

8.3 Error Recovery

Read IRQ_STAT to identify every pending source, service each event, and clear only the corresponding WIC bits. If IRQ remains asserted, re-read IRQ_STAT because a new hardware event may have won a concurrent clear. Reload DATE and TIME after power loss, reset, or an interruption that makes the



software time source authoritative.

9 Verification

9.1 Verification Environment

The reusable UVM environment verifies the APB programming model and IP-specific datapath behavior at the integration top.

9.1.1 Top Testbench

The top testbench generates PCLK and reset, instantiates the DUT and APB interface, publishes the virtual interface, and starts the selected UVM test.

9.1.2 APB Agent

The active APB agent contains the sequencer, driver, and monitor. Sequences describe software-visible transfers while the driver implements APB setup and access phases.

9.1.3 Monitor

The monitor reconstructs completed APB transactions and publishes them through an analysis port without driving DUT signals.

9.1.4 Scoreboard

The scoreboard maintains the reference programming model, checks responses and data, and samples functional coverage.

9.2 Verification Guide

The repository contains a UVM 1.2 environment around `apb_rtc_wrapper`. The testbench overrides `PCLK_FREQ_HZ` to 100 to shorten calendar tests. The register smoke sequence checks reset/readback behavior and `VERSION`. Directed sequences cover leap-day and non-leap February rollover, alarm generation, second-tick IRQ assertion, and `IRQ_STAT` write-one-to-clear behavior. Release sign-off should extend this baseline with all month ends, 23:59:59 year rollover, century rules (including 2100 and 2400), invalid load correction, time-only versus date-qualified alarms, simultaneous clear/set priority, interrupt masking, `PSTRB` behavior, unmapped addresses, reset interruption, and prescaler boundary values.



9.3 Coverage and Sign-Off

Sign-off evidence

Release evidence shall include compile, lint, CDC, RDC, simulation regression, functional coverage, and code coverage summaries generated from the tagged RTL revision.

